

Engineering Progress Tracker

Dual-MCU IoT Sensor Node

STM32F411 Black Pill + ESP32 DevKit V1 · Fedora Linux

Print · Check boxes with a pen · Date each completed item

Project Overview

Total tasks across all 7 phases: **215**. Work through phases sequentially — do not advance until the exit gate is cleared.

Phase	Name	Duration	Tasks	Gates
0	Foundation	~3 weeks	30	1
1	Peripheral Drivers	~7 weeks	56	9
2	FreeRTOS + ESP32 Baseline	~6 weeks	30	2
3	Inter-MCU Protocol	~5 weeks	23	4
4	Bootloader + OTA Pipeline	~8 weeks	29	4
5	System Integration	~4 weeks	19	1
6	PCB Design + Fabrication	~6 weeks	28	1

■ Regular task

◆ Exit gate (must pass before advancing)

◆ Git tag

● Git commit

PHASE 0 Foundation

~3 weeks

Entry Intermediate C, basic Arduino experience.

Exit Working Makefile, firmware flashed via ST-Link, ARM memory map understood, GitHub repo live.

0.1 — C LANGUAGE HARDENING

- Read Beej's Guide to C, Chapters 7–8 (Pointers & Memory) — beej.us/guide/bgc/
- Practice bitwise set/clear/toggle on paper before touching the MCU
- Write a struct-pointer-casting demo: cast `uint32_t` to `struct*`, access fields
- Write a function-pointer typedef and a 3-entry dispatch table
- Explain volatile to yourself — what breaks without it in a status-register poll
- Complete Exercise Set 0.1 (7 exercises in Exercise Book)

0.2 — TOOLCHAIN SETUP ON FEDORA

- `sudo dnf install arm-none-eabi-gcc-cs arm-none-eabi-newlib arm-none-eabi-gdb openocd make git`
- Verify: `arm-none-eabi-gcc --version` returns a version line
- Create `/etc/udev/rules.d/70-stlink.rules` with both ST-Link idVendor entries
- `sudo udevadm control --reload-rules && sudo udevadm trigger`
- Plug ST-Link — verify: `lsusb | grep STMicro` shows device
- Write Makefile from template (type it — do not copy-paste)
- Write minimal `main.c` and startup stub — `make` builds `firmware.elf` + `firmware.bin`
- Flash via OpenOCD — see 'verified' in output
- Complete Exercise Set 0.2 (5 exercises in Exercise Book)

0.3 — DATASHEETS

- Download RM0383 (STM32F411 Reference Manual) from st.com
- Download STM32F411 Datasheet from st.com
- Download ARM Cortex-M4 User Guide (DUI-0553) from developer.arm.com
- Download ESP32 Technical Reference Manual from espressif.com
- Read RM0383 Section 2.3 (Memory Map) — find GPIOA, USART1, I2C1, SPI1 addresses
- Read RM0383 Section 6 (RCC) — understand clock tree
- Read RM0383 Sections 8.3–8.4 (GPIO MODER, OTYPER, OSPEEDR, PUPDR)
- Complete Exercise Set 0.3 (5 exercises in Exercise Book)

0.4 — REPOSITORY

- Create GitHub repo: `dual-mcu-iot-node`
- Push directory skeleton with `.gitkeep` files
- Write `README.md`: one honest paragraph describing the project
- Write `docs/build.md`: toolchain install + compile + flash commands
- ◆ **GATE: LED blinks at 1Hz compiled and flashed via your own Makefile**
- ◆ **git tag -a v0.0-toolchain-verified && git push origin v0.0-toolchain-verified**

Date: _____

Date: _____

● commit: chore(repo): initial project skeleton and verified toolchain

Date: _____

PHASE 1 Peripheral Drivers

~7 weeks

Entry	Phase 0 complete. Makefile works, OpenOCD flashes, RM0383 downloaded.
Exit	UART (ISR ring buffer + printf), GPIO, I2C (BMP280 + SSD1306), SPI, Timer IC (HC-SR04), DHT22 — all tested.

1.1 — GPIO + RCC

■	Read RM0383 Section 6 (RCC) and Section 8 (GPIO) completely	_____
■	Implement <code>gpio_init()</code> with RCC clock enable, MODER, OTYPER, OSPEEDR, PUPDR	_____
■	Implement <code>gpio_set/clear/toggle</code> using BSRR only (not ODR)	_____
■	Implement <code>gpio_read()</code> via IDR	_____
■	Write <code>gpio_test.c</code> — RGB LEDs cycling all combinations	_____
■	Complete Exercise Set 1.1 (5 exercises)	_____
◆	GATE: All 3 RGB LEDs individually controllable; register dump shows correct MODER	Date: _____
●	commit: feat(gpio): register-level GPIO driver with BSRR	Date: _____

1.2 — UART DRIVER

■	Read RM0383 Section 30 (USART) completely — BRR, CR1, SR registers	_____
■	Calculate BRR for 115200 baud on USART2 (APB1=42MHz) — verify on paper	_____
■	Implement <code>uart_init()</code> , <code>uart_write_byte()</code> , <code>uart_write_buf()</code> — polling TX	_____
■	Verify: 'UART OK\r\n' appears in minicom at correct baud	_____
■	Implement <code>_write()</code> syscall override — <code>printf</code> retargeting to UART	_____
■	Implement 256-byte circular ring buffer (power-of-2 masking)	_____
■	Implement <code>USART2_IRQHandler</code> — stores bytes in ring buffer	_____
■	Implement <code>uart_rx_available()</code> and <code>uart_rx_get()</code>	_____
■	Implement <code>HardFault_Handler</code> — print PC and LR over UART then halt	_____
■	TEST <code>HardFault</code> : dereference NULL intentionally — verify UART output	_____
■	Write <code>uart_test.c</code> — 10,000-byte loopback test	_____
■	Complete Exercise Set 1.2 (5 exercises)	_____
◆	GATE: 10,000-byte loopback zero errors; HardFault prints PC/LR on null deref	Date: _____
●	commit: feat(uart): polling TX, ISR RX ring buffer, printf, HardFault handler	Date: _____

1.3 — SYSTICK TIMER

■	Implement <code>systick_init()</code> , <code>SysTick_Handler</code> , <code>systick_get_ms()</code> , <code>systick_delay_ms()</code>	_____
◆	GATE: LED toggles at exactly 1Hz — verify with phone timer over 60 seconds	Date: _____

1.4 — I2C + BMP280 + SSD1306

■	Read RM0383 Section 27 (I2C) completely — SR1/SR2 state machine, CCR	_____
■	Calculate CCR for 100kHz standard mode with PCLK1=42MHz	_____
■	Implement <code>i2c_init()</code> , <code>i2c_start/stop</code> , <code>i2c_write/read_byte</code> with timeouts	_____
■	Implement <code>i2c_bus_recovery()</code> — 9 SCL clock pulses via GPIO bit-bang	_____
◆	GATE: BMP280 WHO_AM_I register (0xD0) returns 0x60	Date: _____

■	Implement BMP280 full driver — compensation formula, temp + pressure	_____
■	Implement SSD1306 init sequence + 5×7 font text rendering	_____
■	Complete Exercise Set 1.4 (5 exercises)	_____
✦	GATE: 100 consecutive BMP280 reads zero errors; SSD1306 shows live temp	Date: _____
●	commit: feat(i2c): driver with bus recovery + BMP280 + SSD1306	Date: _____

1.5 — SPI DRIVER

■	Read RM0383 Section 28 (SPI) completely	_____
■	Implement spi_init() — mode 0, software CS, fPCLK/8	_____
■	Implement spi_transfer_byte()	_____
■	Test BMP280 over SPI: WHO_AM_I returns 0x60	_____
■	Complete Exercise Set 1.5 (5 exercises)	_____
✦	GATE: BMP280 chip ID confirmed over SPI	Date: _____
●	commit: feat(spi): SPI master mode 0, BMP280 SPI verified	Date: _____

1.6 — TIMER IC + HC-SR04

■	Read RM0383 Section 18 (Timers) — Input Capture sub-section	_____
■	Wire HC-SR04 echo line through 8-channel logic level shifter (5V→3.3V)	_____
■	Implement TIM3 Input Capture at 1MHz (1μs resolution), both edges	_____
■	Implement HC-SR04 trigger (10μs pulse) and distance calculation (÷58)	_____
■	Complete Exercise Set 1.6 (5 exercises)	_____
✦	GATE: HC-SR04 reads 30cm target within ±5% over 20 consecutive readings	Date: _____
●	commit: feat(hcsr04): timer input capture + distance calculation	Date: _____

1.7 — DHT22 SINGLE-WIRE

■	Read DHT22 datasheet Section 5 — single-bus timing protocol	_____
■	Wire 4.7kΩ pull-up on DHT22 data line to 3.3V	_____
■	Implement start sequence, Timer IC bit decoding, checksum verification	_____
■	Complete Exercise Set 1.7 (5 exercises)	_____
✦	GATE: DHT22 humidity within ±5% of reference; checksum passes every read	Date: _____
✦	GATE: 12-hour continuous sensor polling — zero HardFaults in log	Date: _____
◆	git tag v0.1-peripheral-drivers	Date: _____
●	commit: feat(dht22): 1-wire driver via timer input capture + checksum	Date: _____

PHASE 2 FreeRTOS + ESP32 Baseline

~6 weeks

Entry	Phase 1 complete. All drivers tested and committed.
Exit	FreeRTOS 6-task architecture running with sensor data in queues. ESP32 publishing to MQTT broker visible in MQTT Explorer.

2A — FREERTOS ON STM32

■	Read FreeRTOS Book Chapters 1–5 — free PDF at freertos.org	_____
■	git submodule add FreeRTOS-Kernel into stm32/rtos/	_____
■	Add portable/GCC/ARM_CM4F/port.c and heap_4.c to Makefile	_____
■	Configure FreeRTOSConfig.h — CPU Hz, tick 1000Hz, heap 16KB, stack overflow mode 2	_____
■	Define configASSERT() to uart_printf file+line then halt	_____
■	Create task_sensor.c — SensorAcquisition (HIGH priority, 512 words)	_____
■	Create task_packager.c — DataPackager (MEDIUM, 256 words)	_____
■	Create task_uart_tx.c — UARTTransmit (MEDIUM, 256 words)	_____
■	Create task_uart_rx.c — UARTReceive (HIGH, 512 words)	_____
■	Create task_cmd.c — CommandHandler (MEDIUM, 256 words)	_____
■	Create task_health.c — SystemHealth (LOW, 128 words)	_____
■	Define SensorDataQueue (10 items), TxQueue (10), CmdQueue (5)	_____
■	TEST: Intentionally overflow a stack — verify hook prints task name	_____
■	Run vTaskGetRunTimeStats() — verify all 6 tasks scheduled	_____
■	Complete Exercise Set 2A (6 exercises)	_____
◆	GATE: All tasks running, sensor data in queues, no overflow in 24-hour run	Date: _____
◆	git tag v0.2-freertos-tasks	Date: _____
●	commit: feat(freertos): 6-task architecture, sensor data pipeline	Date: _____

2B — ESP32 + WIFI + MQTT

■	Install ESP-IDF on Fedora — follow docs.espressif.com Linux setup	_____
■	Verify: idf.py --version returns version string	_____
■	Configure dual OTA partition table (ota_0 + ota_1) in partitions.csv	_____
■	Implement WiFi station mode — handle WIFI_EVENT_STA_CONNECTED + IP_EVENT_STA_GOT_IP	_____
■	Implement MQTT client — connect to broker.hivemq.com port 1883	_____
■	Implement MQTT publish — send JSON to device/001/test every 5 seconds	_____
■	Implement MQTT subscribe — subscribe to device/001/cmd	_____
■	Verify with MQTT Explorer on laptop — see published messages	_____
■	Complete Exercise Set 2B (4 exercises)	_____
◆	GATE: WiFi connects; MQTT messages visible in MQTT Explorer	Date: _____
◆	git tag v0.2-esp32-mqtt-baseline	Date: _____
●	commit: feat(esp32): WiFi + MQTT + dual OTA partition table	Date: _____

PHASE 3 Inter-MCU Protocol

~5 weeks

Entry	Phase 2 complete. FreeRTOS running. ESP32 on MQTT.
Exit	Framed UART protocol with CRC-16 exchanging data between MCUs. Bit errors detected, NAK'd, retransmitted.

3.1 — PROTOCOL SPECIFICATION

■	Write protocol/protocol_spec.md — complete frame format table	_____
■	Document byte stuffing rules (0xAA, 0x55, 0xBB escape sequences)	_____
■	Define full command table (SENSOR_DATA, ACK, NAK, CONFIG_*, OTA_*, PING, PONG)	_____
■	Document timeout and retry policy (500ms NAK window, max 3 retries)	_____
■	Complete Exercise Set 3.1 (4 exercises)	_____

3.2 — CRC + ENCODER (PC FIRST)

■	Implement CRC-16/CCITT table-driven in pure C — compile and run on Linux with gcc	_____
■	Verify CRC against crccalc.com for 5 different byte sequences — all must match	_____
■	Implement byte stuffing encoder and decoder — test round-trip	_____
■	Implement frame encoder: build_frame(cmd, seq, payload, len)	_____
■	Complete Exercise Set 3.2 (4 exercises)	_____

3.3 — PARSER STATE MACHINE

■	Implement 9-state parser (IDLE/LEN_HI/LEN_LO/SEQ/CMD/PAYLOAD/CRC_HI/CRC_LO/EOF)	_____
■	Implement protocol_feed_byte() — non-blocking, one byte per call	_____
■	Unit test parser on PC with known byte sequences	_____
■	Inject single bit flip — verify CRC detects it and frame is discarded	_____
■	Port CRC + encoder + parser to STM32	_____
■	Port CRC + encoder + parser to ESP32	_____

3.4 — VALIDATION + LOGIC ANALYSER

■	Buy logic analyser (Cypress FX2 clone, ~£1200) + install Sigrok/PulseView	_____
◆	GATE: STM32 SENSOR_DATA frame visible decoded in Sigrok	Date: _____
◆	GATE: ESP32 decodes frame, publishes JSON to MQTT — visible in MQTT Explorer	Date: _____
◆	GATE: Inject bit error — CRC detected, NAK sent, retransmit occurs	Date: _____
◆	GATE: CONFIG_SET from ESP32 — STM32 ACKs and changes poll interval	Date: _____
◆	git tag v0.3-inter-mcu-protocol	Date: _____
●	commit: feat(protocol): framed UART CRC-16 protocol, byte stuffing validated	Date: _____

PHASE 4 Bootloader + OTA Pipeline

~8 weeks

Entry	Inter-MCU protocol passing all acceptance tests.
Exit	Laptop MQTT push triggers STM32 to receive, flash, verify, and reboot into new firmware. Zero physical access required.

4.1 — FLASH LAYOUT + LINKER SCRIPTS

■	Map STM32F411 flash sectors on paper: Sector 0–7, addresses, sizes	_____
■	Write stm32/bootloader/bootloader.ld — ORIGIN=0x08000000, LENGTH=32K	_____
■	Write stm32/app/application.ld — ORIGIN=0x08010000, LENGTH=448K	_____
■	Add SCB->VTOR = 0x08010000 to SystemInit() in application	_____
■	Verify both .map files: .text sections at correct start addresses	_____
■	Complete Exercise Sets 4.1–4.2 (7 exercises)	_____

4.2 — BOOTLOADER CORE

■	Create stm32/bootloader/ as separate Make project with own Makefile	_____
■	Implement flash_unlock(), flash_sector_erase(), flash_write_word(), flash_lock()	_____
■	Implement flash_crc32(start_addr, length) over application region	_____
■	Define config sector layout: OTA flag at 0x08008000, app CRC at 0x08008004	_____
■	Implement bootloader_check_ota_flag() and bootloader_validate_app()	_____
■	Implement jump_to_app(0x08010000) — all 7 steps in correct order	_____
■	Implement bootloader main() — check flag → validate → jump or wait	_____
■	Complete Exercise Set 4.3 (3 exercises)	_____
✦	GATE: Bootloader jumps to v1 app — LED blinks at 1Hz confirming jump	Date: _____
✦	GATE: Corrupt app CRC — bootloader must NOT jump, must wait	Date: _____
◆	git tag v0.4-bootloader	Date: _____
●	commit: feat(bootloader): flash write, CRC validation, app jump sequence	Date: _____

4.3 — OTA PIPELINE

■	Implement ESP32 MQTT subscriber: device/001/ota/start + device/001/ota/chunk	_____
■	Implement ESP32 → STM32 OTA_DATA frame forwarding via protocol	_____
■	Implement STM32 OTA_START, OTA_DATA, OTA_END handlers	_____
■	Write scripts/ota_push.py — reads .bin, publishes to MQTT in sequence	_____
■	Build firmware_v1.bin (1Hz blink) and firmware_v2.bin (4Hz blink)	_____
■	TEST: python ota_push.py firmware_v2.bin — LED changes to 4Hz	_____
■	TEST power loss: cut power mid-OTA — verify bootloader refuses to jump	_____
✦	GATE: Full end-to-end OTA from laptop to STM32 running new firmware	Date: _____
✦	GATE: Power-loss during OTA — bootloader correctly refuses partial image	Date: _____
◆	git tag v0.4-ota-pipeline	Date: _____
●	commit: feat(ota): end-to-end OTA pipeline, MQTT to STM32 flash	Date: _____

PHASE 5 System Integration

~4 weeks

Entry	All subsystems individually verified.
Exit	72-hour soak complete. Zero unexpected resets. OTA mid-soak successful. Watchdog enabled and proven.

5.1 — INTEGRATION BRINGUP

- Connect all subsystems simultaneously — first full system run _____
- Fix any stack overflows: increase in 64-word increments until hook stops _____
- Fix I2C lockup under FreeRTOS load: add bus recovery to SensorAcquisition _____
- Fix MQTT reconnection: ensure fully async in event loop _____
- Add SystemHealth MQTT publish: device/001/health (heap, uptime, wd_resets) _____

5.2 — WATCHDOG + HARDENING

- Implement IWDG: prescaler /64, reload 2047 (~4 second timeout) _____
- Start IWDG before vTaskStartScheduler() _____
- SystemHealth task kicks IWDG every 500ms _____
- TEST: Freeze SystemHealth — verify MCU reboots within 4 seconds _____
- Log reset reason in bootloader via RCC_CSR _____

5.3 — 72-HOUR SOAK

- Flash v1.0 — verify sensor data on MQTT dashboard, note start time _____
- Hour 4: OTA update to v1.1 mid-soak — verify reboot and new firmware _____
- Hour 12: cut WiFi 30min — verify ESP32 reconnects automatically _____
- Hour 24: power cycle STM32 — verify boots into application _____
- Hour 48: power cycle ESP32 — verify MQTT reconnects _____
- Hour 72: health topic shows uptime ~72hr, watchdog_resets = 0 _____
- ◆ **GATE: 72-hour soak complete, zero unexpected resets** Date: _____
- ◆ **git tag v1.0-system-integration** Date: _____
- **commit: feat(integration): IWDG, health MQTT, 72hr soak verified** Date: _____

PHASE 6 PCB Design + Fabrication

~6 weeks

Entry	v1.0 integration complete. All component electrical behaviour understood.
Exit	Fabricated PCB running all firmware. OTA update verified on custom hardware. 24-hour soak passed.

6.1 — KICAD SETUP

■	flatpak install flathub org.kicad.KiCad — verify KiCad 8.x launches	_____
■	Complete KiCad Schematic Editor tutorial at docs.kicad.org	_____
■	Complete KiCad PCB Editor tutorial at docs.kicad.org	_____

6.2 — SCHEMATIC

■	Add STM32F411CEU6 and ESP32 schematic symbols	_____
■	Add 100nF decoupling cap on every VDD/VDDA pin	_____
■	Add 10µF bulk caps per power domain	_____
■	Add 100nF NRST filter cap, 10kΩ BOOT0 pull-down	_____
■	Add 4-pin SWD header (SWDIO, SWDCLK, GND, 3V3)	_____
■	Add ESP32 EN RC filter (10kΩ + 1µF)	_____
■	Add I2C pull-ups (4.7kΩ on SDA and SCL)	_____
■	Add AMS1117-3.3 power section with filter caps	_____
■	Add sensor connectors (JST or pin headers)	_____
■	Run ERC — resolve all errors	_____

6.3 — LAYOUT + FABRICATION

■	Assign footprints to all components	_____
■	Add ground plane on Layer 2 (solid copper, all GND nets)	_____
■	Place decoupling caps within 1mm of VDD pins	_____
■	Route all traces — no right angles, 45° bends	_____
■	Run DRC — resolve all errors	_____
■	Export Gerbers — upload to JLCPCB/PCBWay, verify 3D preview	_____
■	Order 5 boards	_____
■	Boards arrived — power through USB current meter, check draw	_____
■	Flash via SWD — verify ST-Link connects	_____
■	Run Phase 1 driver test suite on PCB	_____
■	OTA update on PCB — verify end-to-end	_____
■	24-hour soak on PCB	_____
◆	GATE: PCB running all firmware, OTA verified on custom hardware	Date: _____
◆	git tag v1.1-pcb-bringup	Date: _____
●	commit: feat(pcb): KiCad schematic + layout, fabricated board operational	Date: _____